



香港中文大學(深圳)

The Chinese University of Hong Kong, Shenzhen

A Survey of Streaming Real-Time Speech Generation: From Codec Language Models to System-Level Serving

Zihao Fang

房子皓

CSC 6032

The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Real-time speech generation is becoming a core interface for intelligent cockpits, virtual live streaming, and end-to-end spoken dialogue systems. Its difficulty is not only to synthesize natural speech, but also to produce it incrementally under strict latency, context, and serving-cost constraints. Following the order of the presentation, this survey reviews three representative papers as a continuous technical trajectory. VALL-E first replaces continuous mel-spectrogram regression with neural codec tokens and formulates zero-shot text-to-speech as conditional language modeling. CosyVoice 2 extends this idea into a scalable streaming architecture by combining supervised semantic speech tokens, a pre-trained large language model, and chunk-aware causal flow matching. vLLM/PagedAttention then addresses the deployment bottleneck of autoregressive large-model serving by paging the KV cache and enabling higher-throughput continuous batching.

The central argument of this review is that streaming real-time speech generation requires progress at three coupled levels: representation, model architecture, and serving system. Discrete speech tokens provide the native sequential interface needed by autoregressive generation; large language models improve long-context understanding and conversational naturalness; and operating-system-inspired memory management makes high-concurrency low-latency serving practical. By comparing the assumptions, methods, evaluations, and limitations of the three works,

this survey summarizes how modern TTS moves from laboratory-quality synthesis toward commercially deployable interactive speech generation.

摘要

实时语音生成正在成为智能座舱、虚拟直播和端到端语音对话系统中的关键交互能力。它的难点不仅是合成自然语音，还包括在严格延迟、上下文建模和服务成本约束下进行增量生成。本文按照汇报中的叙事顺序，对三篇代表性论文进行综述，并将它们组织为一条连续的技术演进路径：VALL-E 首先将连续梅尔谱回归替换为神经音频编码器产生的离散语音 token，把零样本文本转语音建模为条件语言建模任务；CosyVoice 2 在此基础上引入监督语义语音 token、预训练大语言模型和 chunk-aware causal flow matching，形成可同时支持流式与非流式合成的统一架构；vLLM/PagedAttention 则从系统层面解决自回归大模型服务中的 KV cache 内存碎片和吞吐瓶颈。

本文的核心观点是，流式实时语音生成需要在表示层、模型层和系统层同时取得进展。离散语音 token 为自回归增量生成提供天然接口；大语言模型提升了长上下文理解和对话自然度；借鉴操作系统分页思想的显存管理则使高并发、低延迟部署成为可能。通过比较三项工作的研究动机、方法设计、实验结果与局限性，本文总结了现代 TTS 从实验室高质量合成走向商业化交互式语音服务的关键路径。

Contents

摘要	iii
Contents	iv
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.2 From Offline TTS to Streaming TTS	2
1.3 Related Work	3
1.4 Three Core Challenges	4
1.5 Organization	4
2 VALL-E: Discrete Speech Tokens as the Foundation	6
2.1 Motivation	6
2.2 Original Paper Method Details	7
2.3 Experimental Evidence	8
2.4 Relation to Streaming TTS	9
2.5 Limitations	10

3	CosyVoice 2: LLM-Based Unified Streaming TTS	12
3.1	Motivation	12
3.2	Original Paper Method Details	13
3.3	Chunk-Aware Flow Matching	15
3.4	Experimental Evidence	15
3.5	Relation to Streaming TTS	17
3.6	Role in the Technical Chain	17
4	vLLM and PagedAttention: System Support for Real-Time Serving	18
4.1	Why a Serving Paper Matters for TTS	18
4.2	Original Paper Method Details	19
4.3	Continuous Batching and Scheduling	21
4.4	Experimental Evidence	21
4.5	Relation to Streaming TTS	21
5	Conclusion	24
5.1	Synthesis	24
5.2	Future Work	25
	Bibliography	26

List of Figures

2.1	Overview of VALL-E from the original paper. The system replaces the traditional phoneme-to-mel-to-waveform pipeline with phoneme-to-discrete-code-to-waveform generation. The acoustic prompt specifies the target speaker and acoustic condition.	7
3.1	Overall architecture of CosyVoice 2 from the original paper. The system separates supervised semantic speech tokenization, unified text-speech language modeling, and chunk-aware flow matching.	13
3.2	Unified text-speech LM in CosyVoice 2. Non-streaming synthesis concatenates text before speech tokens, while streaming synthesis interleaves text and speech tokens at a fixed ratio.	14
4.1	KV cache memory management in existing serving systems. The original vLLM paper highlights reserved memory, internal fragmentation, and external fragmentation caused by contiguous allocation.	19
4.2	PagedAttention stores logically consecutive KV cache blocks in non-contiguous physical memory and uses a block table to fetch them during attention computation.	20

List of Tables

- 2.1 Representative objective results reported by VALL-E on LibriSpeech. Lower WER is better; higher SPK indicates stronger speaker similarity. 9
- 3.1 Representative LibriSpeech test-clean results from CosyVoice 2. Lower WER and higher NMOS/SS are better. 16
- 3.2 Abridged modular ablation from CosyVoice 2 on SEED test sets. The sequence of rows shows how LLM initialization, dropping speaker embedding, and FSQ contribute to content consistency and speaker similarity. 16
- 4.1 Representative vLLM results relevant to streaming large-model serving. 22

Chapter 1

Introduction

1.1 Background

Text-to-speech (TTS) aims to generate natural speech from text. Modern neural TTS systems have already reached high naturalness in many offline settings, moving from sequence-to-sequence acoustic modeling [Wang et al., 2017; Shen et al., 2018; Li et al., 2019] to faster non-autoregressive and end-to-end systems [Ren et al., 2019, 2021; Kim et al., 2021]. However, interactive applications place a stricter requirement on the generation process. In intelligent cockpits, excessive latency can become a safety risk; in virtual live streaming, every small delay weakens the feeling of co-presence; in end-to-end spoken dialogue, the speech generator must cooperate with the streaming text output of a large language model. Therefore, the target is no longer only “high-quality speech after the full sentence is known”, but low-latency, context-aware, and scalable speech generation.

Following this requirement, the three papers reviewed in this thesis can be read as three consecutive answers to one question: how can speech generation become a native streaming large-model service? VALL-E [Wang et al., 2023] gives the representation-level answer by converting speech into discrete codec tokens and treating TTS as a

language modeling task. CosyVoice 2 [Du et al., 2024] gives the model-level answer by introducing a unified LLM-based architecture for both streaming and non-streaming synthesis. vLLM with PagedAttention [Kwon et al., 2023] gives the system-level answer by improving KV cache management for high-throughput autoregressive serving.

1.2 From Offline TTS to Streaming TTS

Traditional neural TTS systems are usually built around continuous acoustic representations. A representative two-stage pipeline first predicts mel-spectrograms from text and then reconstructs waveforms with a vocoder. Tacotron and Tacotron 2 [Wang et al., 2017; Shen et al., 2018] are classic examples of this acoustic-model-plus-vocoder paradigm, while Transformer TTS [Li et al., 2019] replaces recurrent sequence modeling with self-attention. Later non-autoregressive models such as FastSpeech and FastSpeech 2 [Ren et al., 2019, 2021] improve inference speed by predicting durations and generating acoustic frames in parallel. VITS [Kim et al., 2021] further integrates variational inference, adversarial learning, and neural vocoding into an end-to-end framework.

However, both families are naturally designed for full-segment generation. They prefer complete text input, global duration planning, and stable text-speech alignment. If such models are forced to process text in small chunks, prosody may become unnatural, polyphonic words may be mispronounced, sentence breaks may be wrong, and errors from the frontend, acoustic model, and vocoder may accumulate. The core problem is that speech is strongly contextual: pronunciation, rhythm, pause, speaker style, and emotion often depend on words that appear before and after the current chunk.

1.3 Related Work

The three reviewed papers sit at the intersection of three research streams. The first stream is zero-shot and prompt-based TTS. Earlier multi-speaker systems often relied on speaker adaptation or speaker encoders, while YourTTS [Casanova et al., 2022] made zero-shot multi-speaker TTS more accessible by combining TTS and voice conversion. VALL-E [Wang et al., 2023] changes the framing more radically: instead of extracting a fixed speaker embedding, it uses a short acoustic prompt and predicts discrete codec tokens, making speech synthesis resemble in-context learning.

The second stream is discrete speech representation and speech language modeling. Neural audio codecs such as SoundStream [Zeghidour et al., 2021] and EnCodec [Défossez et al., 2022] make it possible to compress waveform into high-fidelity discrete units. AudioLM [Borsos et al., 2023] demonstrates that language modeling over audio tokens can generate coherent audio continuations. These works prepare the ground for codec language models in TTS, where text controls linguistic content and acoustic prompts control voice characteristics.

The third stream is large-model generative architecture and serving. Transformer self-attention [Vaswani et al., 2017] enables scalable sequence modeling, and large language models such as GPT-3 and PaLM [Brown et al., 2020; Chowdhery et al., 2022] show the power of in-context learning and data/model scaling. In speech generation, recent systems such as Voicebox, Seed-TTS, and F5-TTS [Le et al., 2024; Anastasiou et al., 2024; Chen et al., 2024] combine large-scale training with diffusion, flow matching, or codec-token generation. On the serving side, systems such as Faster-Transformer and Orca [NVIDIA, 2023; Yu et al., 2022] improve Transformer inference, while vLLM [Kwon et al., 2023] specifically targets the KV cache memory bottleneck that becomes critical in autoregressive streaming services.

1.4 Three Core Challenges

The presentation summarized the difficulty of streaming real-time TTS into three challenges. First, there is a **modeling barrier**. Continuous mel-spectrogram frames have strong local dependency and are not as convenient as text tokens for incremental autoregressive generation. A native streaming model needs an intermediate representation that can be generated token by token while still preserving enough speech information.

Second, there is a **context modeling barrier**. Streaming systems cannot always observe the full sentence in advance, but they must still make correct pronunciation and prosody decisions. Traditional pipelines often split text frontend, acoustic modeling, and vocoding into separate modules, which weakens global context modeling and increases accumulated latency.

Third, there is a **deployment barrier**. Large-model TTS inherits the serving problems of autoregressive LLMs: requests have dynamic lengths, KV cache grows during decoding, and high concurrency easily leads to memory fragmentation and latency fluctuation. A model that sounds good in isolation is not sufficient if it cannot meet commercial service-level objectives under concurrent traffic.

1.5 Organization

The remainder of this thesis follows the same logic as the slides. Chapter 2 reviews VALL-E as the foundation of codec-token language modeling for speech. Chapter 3 reviews CosyVoice 2 as a unified LLM-based streaming TTS architecture. Chapter 4 reviews vLLM/PagedAttention as the serving-system mechanism that makes large autoregressive models more practical at scale. The conclusion synthesizes the three levels and discusses remaining open problems.

□ **End of chapter.**

Chapter 2

VALL-E: Discrete Speech Tokens as the Foundation

2.1 Motivation

VALL-E, formally titled *Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers*, starts from a simple but powerful observation: the success of large language models comes from discrete token prediction at scale, while conventional TTS still spends much of its modeling capacity on continuous acoustic regression. The paper therefore asks whether speech synthesis can be reformulated as a language modeling problem over discrete acoustic codes.

This reformulation directly addresses the first challenge in Chapter 1. Instead of predicting mel-spectrogram frames, VALL-E uses an off-the-shelf neural audio codec to transform waveform into discrete codec codes. During inference, a short enrolled recording supplies acoustic prompt codes, while the target text supplies linguistic content. The model then generates the target acoustic codes and sends them to the codec decoder to reconstruct waveform.

2.2 Original Paper Method Details

The VALL-E pipeline can be summarized as

text/phoneme prompt + acoustic prompt
 $\rightarrow$ codec language model $\rightarrow$ codec decoder $\rightarrow$ waveform.

This is different from the older path of text to mel-spectrogram to waveform. The intermediate representation is discrete, so it is natural to train an autoregressive model over token sequences.

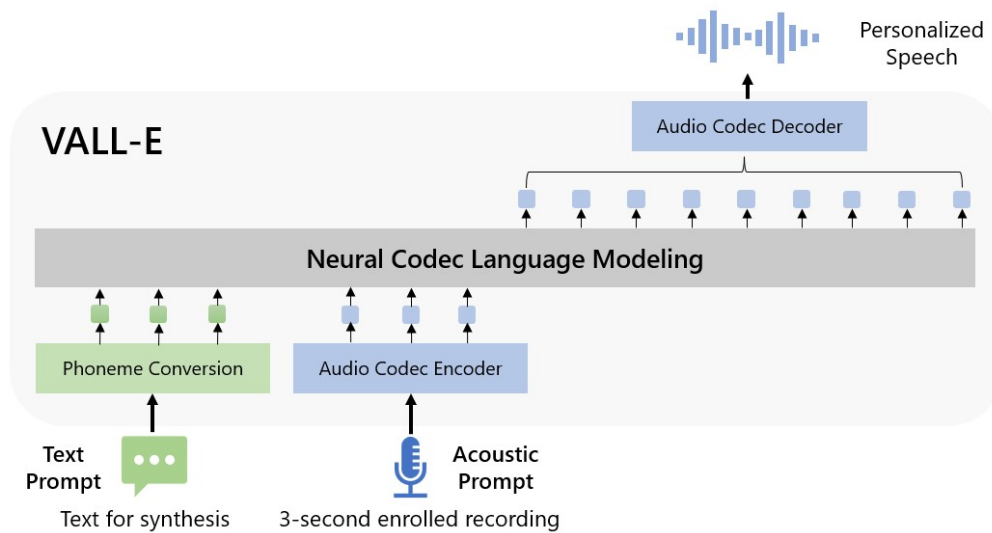


Figure 2.1: Overview of VALL-E from the original paper. The system replaces the traditional phoneme-to-mel-to-waveform pipeline with phoneme-to-discrete-code-to-waveform generation. The acoustic prompt specifies the target speaker and acoustic condition.

The representation choice is the most important technical decision in VALL-E. The model uses neural audio codec codes as the acoustic target, following progress in high-fidelity neural codecs such as SoundStream and EnCodec [Zeghidour et al., 2021; Défossez et al., 2022]. These codes are produced by residual vector quantizers, so one speech segment is represented by several parallel code streams. VALL-E does

not attempt to regress waveform or mel frames directly; it predicts code indices. This makes the acoustic output space closer to text language modeling, where the model selects discrete symbols from a vocabulary.

The conditioning design has two prompts. The phoneme prompt specifies linguistic content. The acoustic prompt is a short enrolled recording, converted into codec codes, that carries speaker identity, speaking style, emotion, and recording environment. During zero-shot inference, the model receives only a few seconds of this enrolled speech and does not require target-speaker fine-tuning. Compared with speaker-embedding methods such as YourTTS [Casanova et al., 2022], this makes the model closer to prompt-based in-context learning in large language models [Brown et al., 2020].

The paper uses two coupled generative parts. The autoregressive model predicts the first quantizer’s codec codes conditioned on phoneme and acoustic prompts. A non-autoregressive model then predicts codes from the remaining quantizers, recovering fine acoustic detail. This design keeps the sequential language-model interface where it matters most, while avoiding the high cost of fully autoregressive generation over all codec streams. In effect, VALL-E treats speech synthesis as coarse-to-fine token completion: first decide the main speech token trajectory, then fill in finer acoustic residual information.

2.3 Experimental Evidence

VALL-E is trained on 60 thousand hours of English speech, hundreds of times larger than many earlier TTS systems. With only a 3-second enrolled recording, it demonstrates zero-shot speaker adaptation through prompting rather than speaker-specific fine-tuning. On LibriSpeech, the paper reports lower word error rate and much higher speaker similarity than YourTTS; in human evaluation, VALL-E reaches a speaker

similarity MOS of 4.38, close to the ground-truth score of 4.50.

Table 2.1: Representative objective results reported by VALL-E on LibriSpeech. Lower WER is better; higher SPK indicates stronger speaker similarity.

Model	WER	SPK
Ground truth	2.2	0.754
GSLM	12.4	0.126
AudioLM	6.0	–
YourTTS	7.7	0.337
VALL-E	5.9	0.580
VALL-E-continual	3.8	0.508

Table 2.1 shows why VALL-E is more than a voice-cloning demonstration. Compared with YourTTS, it improves both content robustness and speaker similarity, even though it uses only a short acoustic prompt at inference. The continual setting further reduces WER because the first three seconds are taken from the ground-truth speech, which gives the model stronger alignment context.

The qualitative findings are especially important for speech generation. Because generation is sampling-based over discrete tokens, VALL-E can produce diverse prosody for the same text, similar in spirit to AudioLM’s use of audio token language modeling for open-ended audio generation [Borsos et al., 2023]. It can also preserve acoustic environment and speaker emotion from the prompt, suggesting that codec tokens carry more than bare linguistic content. This makes the model useful not only for voice cloning, but also for conversational speech where style, environment, and emotion matter.

2.4 Relation to Streaming TTS

VALL-E is not presented as a complete low-latency streaming system, but it provides the first necessary condition for one. Continuous mel-spectrogram modeling

makes chunk-by-chunk generation difficult because future context, duration, and local acoustic continuity are tightly coupled. Discrete codec tokens make speech generation compatible with incremental autoregressive decoding. Once speech is tokenized, the same machinery used by LLMs—prefix conditioning, sampling, context windows, and token-by-token decoding—can be transferred into TTS.

This explains its role in the presentation’s logic. VALL-E solves the “native streaming modeling capability” problem at the representation level. It does not yet solve long-context streaming pronunciation or high-concurrency serving, but it changes the object being generated into something a language model can handle sequentially. CosyVoice 2 and vLLM build on that shift.

2.5 Limitations

VALL-E also exposes the problems that motivate later work. First, its autoregressive phoneme-to-codec model can still produce unclear, missing, or duplicated words due to alignment errors. Second, its data coverage is limited: accents and speaking styles outside the training distribution remain difficult. Third, the architecture still separates different quantizer predictions and does not yet provide a complete unified streaming solution. In the logic of this survey, VALL-E solves the representation bottleneck, but it leaves context-rich streaming synthesis and large-scale serving as open problems.

☐ **End of chapter.**

Chapter 3

CosyVoice 2: LLM-Based Unified Streaming TTS

3.1 Motivation

CosyVoice 2 continues the codec-token direction but focuses on a harder interactive setting. Recent zero-shot TTS models can synthesize high-quality speech, including codec language models, masked codec models, and flow/diffusion-based systems [Wang et al., 2023; Le et al., 2024; Anastassiou et al., 2024; Chen et al., 2024]. Yet most still operate offline: the full input text is known, the whole utterance is generated, and only then is audio returned. For voice chat and other real-time interfaces, this creates a poor user experience. CosyVoice 2 therefore aims to make streaming synthesis nearly lossless compared with offline synthesis while keeping the quality, speaker similarity, and controllability of modern zero-shot TTS.

3.2 Original Paper Method Details

CosyVoice 2 separates semantic and acoustic information. The text-speech language model predicts supervised semantic speech tokens, while a flow matching model converts these tokens, together with speaker and reference-speech conditions, into mel-spectrograms. A vocoder then reconstructs waveform. This hybrid design keeps the alignment and duration advantages of token language modeling while using flow matching for high-quality acoustic generation.

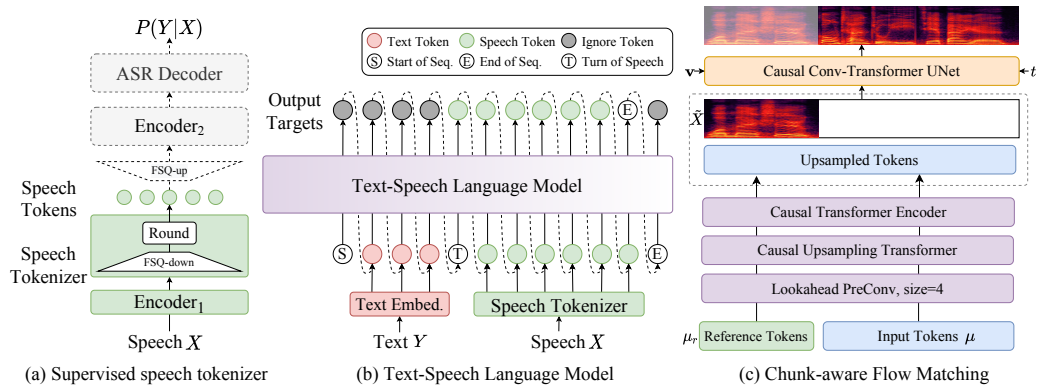


Figure 3.1: Overall architecture of CosyVoice 2 from the original paper. The system separates supervised semantic speech tokenization, unified text-speech language modeling, and chunk-aware flow matching.

The first major component is the speech tokenizer. Instead of ordinary vector quantization, CosyVoice 2 inserts finite scalar quantization (FSQ) into the SenseVoice-Large encoder. FSQ projects encoder representations into a low-rank space, quantizes each scalar dimension with a bounded rounding operation, and converts the resulting vector into a discrete token index. This gives the TTS model semantic speech tokens at a token rate of 25 Hz. The paper emphasizes that FSQ not only uses the codebook more fully, but also reduces speaker entanglement, which addresses a common tension in speech tokenizers: tokens should preserve content and prosody cues without collapsing into speaker identity.

The second component is the text-speech LM. CosyVoice 2 uses Qwen2.5-0.5B as

3.3 Chunk-Aware Flow Matching

The flow matching component is the key to making high-quality acoustic generation compatible with streaming. Diffusion and flow matching have become important alternatives to purely autoregressive codec generation in speech synthesis [Ho et al., 2020; Lipman et al., 2023; Le et al., 2024; Mehta et al., 2024; Chen et al., 2024]. However, ordinary flow matching or diffusion-style TTS tends to wait until all speech tokens are available. CosyVoice 2 instead trains a chunk-aware causal flow matching model with several attention masks: non-causal, full-causal, chunk- M , and chunk- $2M$. These masks cover different latency-quality tradeoffs and allow one model to serve both offline and streaming scenarios.

The model treats multi-step flow estimation as an unfolded causal network. During training, masks with more context implicitly teach masks with less context, creating a self-distillation effect. This is an important model-level answer to the second challenge in Chapter 1: streaming generation loses future context, so the model must be trained to operate under controlled context constraints rather than being forced into streaming after offline training.

3.4 Experimental Evidence

CosyVoice 2 is trained with large-scale multilingual data. The tokenizer uses 200 thousand hours of Chinese and English ASR-style data; the TTS model uses 130 thousand hours of Chinese, 30 thousand hours of English, 4.6 thousand hours of Japanese, and 2.2 thousand hours of Korean. On LibriSpeech test-clean, CosyVoice 2 reports WER of 2.47%, NMOS of 3.96, and speaker similarity of 0.745, outperforming several open-source baselines and its predecessor.

The most relevant result for this survey is the comparison between offline CosyVoice 2 and streaming CosyVoice 2-S. On ordinary Chinese and English test sets, streaming

Table 3.1: Representative LibriSpeech test-clean results from CosyVoice 2. Lower WER and higher NMOS/SS are better.

Model	WER (%)	NMOS	SS
Human	2.66	3.84	0.697
CosyVoice	2.89	3.93	0.743
CosyVoice 2	2.47	3.96	0.745
CosyVoice 2-S	2.45	3.90	0.751

Table 3.2: Abridged modular ablation from CosyVoice 2 on SEED test sets. The sequence of rows shows how LLM initialization, dropping speaker embedding, and FSQ contribute to content consistency and speaker similarity.

Model	test-zh		test-en		test-hard	
	CER	SS	WER	SS	WER	SS
CosyVoice	3.63	0.775	4.29	0.699	11.75	0.755
+ LLM initialization	2.96	0.808	4.57	0.730	9.94	0.789
+ Drop speaker embedding	2.56	0.804	3.81	0.740	9.66	0.778
+ FSQ (CosyVoice 2)	1.45	0.806	2.57	0.736	6.83	0.776

mode is nearly lossless; the main degradation appears on hard cases involving repetition, tongue twisters, and other challenging inputs. This supports the slide’s claim that LLM-based context modeling is a key breakthrough for streaming TTS, while also showing that difficult text still stresses limited-context generation.

Tables 3.1 and 3.2 show that the paper’s improvements are not isolated tricks. LLM initialization improves contextual alignment, dropping speaker embedding reduces harmful information leakage, and FSQ gives a large additional gain in content consistency. The streaming model preserves most of the offline model’s quality, which is the central evidence for CosyVoice 2 as a streaming TTS architecture.

3.5 Relation to Streaming TTS

CosyVoice 2 addresses the second slide challenge: streaming scenarios degrade performance because models lack enough context. It does so by making streaming part of the training objective rather than a post-processing trick. The text-speech LM learns when to emit speech tokens and when to wait for more text. The chunk-aware flow matching model learns how much future acoustic context it can use under different latency settings.

This also makes CosyVoice 2 naturally compatible with LLM-based voice chat. If a dialogue LLM emits text tokens gradually, CosyVoice 2 can begin speech-token generation after only a small text prefix. Its latency analysis explicitly separates LLM text-token delay, speech-token generation delay, flow matching delay, and vocoder delay, which matches the end-to-end streaming pipeline described in the presentation.

3.6 Role in the Technical Chain

Compared with VALL-E, CosyVoice 2 is less about proving that codec-token TTS works and more about engineering it into a general streaming architecture. It adds stronger semantic tokens, pre-trained LLM context modeling, unified streaming/non-streaming training, and chunk-aware acoustic decoding. However, it also makes the serving problem more serious: once TTS becomes an LLM-style autoregressive service, its latency and cost depend heavily on the efficiency of large-model inference. That leads naturally to vLLM and PagedAttention.

□ **End of chapter.**

Chapter 4

vLLM and PagedAttention: System Support for Real-Time Serving

4.1 Why a Serving Paper Matters for TTS

At first glance, vLLM is not a speech synthesis paper. It is a serving system for large language models. In the logic of this survey, however, it completes the final step from model capability to usable real-time service. VALL-E and CosyVoice 2 both rely on autoregressive token generation. Their streaming behavior therefore inherits the same serving difficulty as LLMs: many requests arrive concurrently, each request has a different prompt length and output length, and every decoding step depends on the accumulated KV cache. This problem has been studied by earlier Transformer serving systems such as FasterTransformer and Orca [NVIDIA, 2023; Yu et al., 2022], but vLLM focuses specifically on the memory management bottleneck.

The slides describe this as an inference-level bottleneck. Static VRAM allocation cannot adapt to dynamic streaming requests. It creates fragmentation, limits batch size, and causes latency fluctuations. For commercial TTS, this matters directly: a system may synthesize good speech for one user but still fail under high concurrency

if GPU memory is wasted by KV cache allocation.

4.2 Original Paper Method Details

In autoregressive Transformer serving, the model generates tokens one by one [Vaswani et al., 2017]. To avoid recomputing attention over all previous tokens, the server stores the key and value tensors for each generated position. This KV cache grows as generation proceeds. For large models such as OPT and PaLM [Zhang et al., 2022; Chowdhery et al., 2022], it can occupy a large fraction of available GPU memory. The vLLM paper gives a concrete example: for OPT-13B, the KV cache of one token can require about 800 KB, and one request may require up to 1.6 GB if it reaches 2048 tokens.

Existing systems usually allocate contiguous memory chunks for each request, often according to the maximum possible sequence length. This causes three kinds of waste: reserved but currently unused slots, internal fragmentation from over-provisioning, and external fragmentation from the allocator. The paper reports that existing systems may use only 20.4%–38.2% of KV cache memory for actual token states in their profiling.

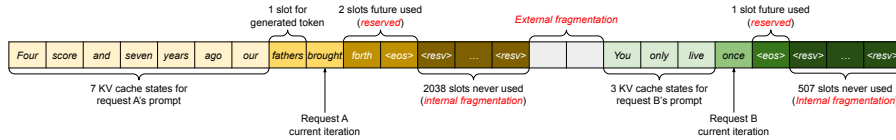


Figure 4.1: KV cache memory management in existing serving systems. The original vLLM paper highlights reserved memory, internal fragmentation, and external fragmentation caused by contiguous allocation.

PagedAttention adapts the virtual memory idea from operating systems. Instead of storing a request’s KV cache in one contiguous tensor, it partitions the KV cache into fixed-size blocks. A logical block table maps each sequence’s logical token blocks

to physical blocks in GPU memory. As a result, logical continuity no longer requires physical continuity.

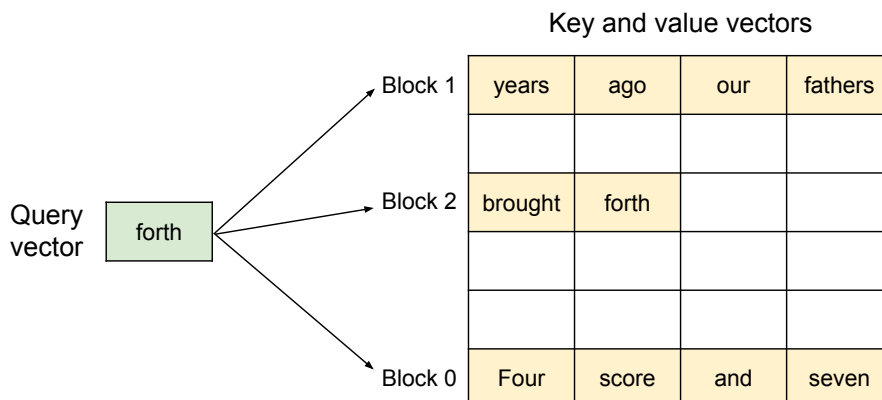


Figure 4.2: PagedAttention stores logically consecutive KV cache blocks in non-contiguous physical memory and uses a block table to fetch them during attention computation.

This design has three effects. First, memory is allocated on demand, so reserved unused space is reduced. Second, all physical blocks have the same size, which removes external fragmentation. Third, blocks can be shared across sequences, enabling efficient parallel sampling, beam search, and shared-prefix workloads. Copy-on-write is used when a shared block must be modified.

vLLM builds a full serving engine around this abstraction. The centralized scheduler tracks requests, sequence groups, and block tables. A KV cache manager allocates and frees GPU blocks. For decoding algorithms that fork multiple continuations from the same prompt, vLLM shares blocks and uses reference counts. When a shared block must be extended or modified, the system applies copy-on-write and copies only the necessary block.

4.3 Continuous Batching and Scheduling

PagedAttention is paired with a serving engine, vLLM. The scheduler can add new requests between decoding iterations, remove completed requests, and manage preemption when memory is exhausted. This iteration-level scheduling is essential for real-time workloads because incoming requests should not wait for an entire previous batch to finish.

The paper also supports preemption. If GPU blocks are exhausted, vLLM may temporarily evict a whole sequence group. It can recover by swapping KV blocks to CPU memory or by recomputing the KV cache later. The all-or-nothing sequence-level policy is chosen because the attention pattern of autoregressive generation usually needs the whole prefix together.

4.4 Experimental Evidence

Across OPT and LLaMA models on ShareGPT and Alpaca workloads, vLLM improves serving throughput by 2–4 times compared with state-of-the-art systems at comparable latency, with even larger gains in some settings against less flexible baselines. For basic sampling on ShareGPT, vLLM sustains 1.7–2.7 times higher request rates than an oracle version of Orca and 2.7–8 times higher than a maximum-reservation version. For beam search, block sharing gives large memory savings, reported as 37.6%–55.2% on Alpaca and 44.3%–66.3% on ShareGPT.

4.5 Relation to Streaming TTS

The relationship between vLLM and streaming TTS is indirect but essential. A streaming TTS model has stricter latency requirements than a general chatbot: it must produce the first audio package quickly and continue producing chunks smoothly enough

Table 4.1: Representative vLLM results relevant to streaming large-model serving.

Scenario	Reported result
Overall serving throughput	$2\text{--}4\times$ improvement over state-of-the-art LLM serving systems at comparable latency
Basic sampling on ShareGPT	$1.7\text{--}2.7\times$ higher request rates than Orca (Oracle); $2.7\text{--}8\times$ higher than Orca (Max)
Parallel sampling memory sharing	6.1%–9.8% memory saving on Alpaca; 16.2%–30.5% on ShareGPT
Beam search memory sharing	37.6%–55.2% memory saving on Alpaca; 44.3%–66.3% on ShareGPT
Shared-prefix translation	$1.67\times$ higher throughput with one-shot prefix; $3.58\times$ with few-shot prefix

for playback. However, once the acoustic generator is built on autoregressive token decoding, it faces the same memory-bound serving regime as LLMs. The KV cache grows with each generated speech token, and different users may have very different prompt lengths, output lengths, and speaking rates.

PagedAttention is therefore the system-level counterpart to codec speech tokens. VALL-E and CosyVoice 2 make speech look like a token sequence; vLLM makes many such token sequences feasible to serve concurrently. In a complete real-time speech stack, an LLM may stream text to a TTS LM, the TTS LM may stream speech tokens to a flow matching/vocoder backend, and PagedAttention-like memory management can stabilize the autoregressive parts of that pipeline.

For streaming TTS, the relevance is direct. If speech tokens are generated by an LLM-style decoder, first-package latency and sustained throughput depend on how many concurrent decoding streams can fit in GPU memory. PagedAttention therefore provides the system-level foundation for turning models like CosyVoice 2 into high-concurrency services. In the three-paper chain, it solves the deployment bottleneck that representation and model innovations alone cannot remove.

☐ **End of chapter.**

Chapter 5

Conclusion

5.1 Synthesis

This survey follows a three-level path. At the representation level, VALL-E shows that speech can be discretized into codec tokens and generated through language modeling. This is the conceptual breakthrough that makes native incremental generation plausible. At the model level, CosyVoice 2 shows how LLM-based context modeling, supervised semantic tokens, and chunk-aware flow matching can make streaming synthesis nearly lossless compared with offline generation. At the system level, vLLM/PagedAttention shows how autoregressive generation can be served under high concurrency by managing KV cache with paging, sharing, and continuous batching.

The combined lesson is that streaming real-time TTS is not a single-model problem. A practical system must align three interfaces: text and speech must share a token-like representation; the model must understand long conversational context while emitting short chunks; and the serving engine must keep latency stable as request lengths and concurrency change. If any one of these layers is weak, the whole real-time experience breaks.

5.2 Future Work

Several problems remain open. First, streaming models still need stronger robustness on hard text, including repetitions, tongue twisters, code-switching, and polyphonic characters. Second, multilingual and accent coverage remains uneven, as shown by both VALL-E’s discussion of accent speakers and CosyVoice 2’s Japanese and Korean benchmark gap. Third, controllability and safety must advance together: zero-shot voice cloning and emotion transfer are useful, but they also increase risks of impersonation and misuse.

Finally, there is room for deeper co-design between TTS models and serving systems. vLLM optimizes generic LLM serving, but speech generation has extra timing constraints such as first-package latency, audio chunk size, vocoder scheduling, and synchronization with streaming text output. Future real-time speech systems may need schedulers that understand both token-level decoding and audio-level playback deadlines.

□ **End of chapter.**

Bibliography

- Anastassiou, P., Chen, J., Chen, J., Chen, Y., Cong, Z., Deng, L., Ding, C., Gao, L., Gong, M., et al. (2024). Seed-tts: A family of high-quality versatile speech generation models. *arXiv preprint arXiv:2406.02430*.
- Borsos, Z., Marinier, R., Vincent, D., Kharitonov, E., Pietquin, O., Sharifi, M., Roblek, D., Teboul, O., Grangier, D., Tagliasacchi, M., and Zeghidour, N. (2023). Audioldm: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, pages 2523–2533.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, pages 1877–1901.
- Casanova, E., Weber, J., Shulby, C. D., Junior, A. C., Gölge, E., and Ponti, M. A. (2022). Yourtts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone. In *International Conference on Machine Learning*, pages 2709–2720.
- Chen, Y., Niu, Z., Ma, Z., Deng, K., Wang, C., Zhao, J., Yu, K., and Chen, X. (2024). F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching. *arXiv preprint arXiv:2410.06885*.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. (2022). Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*.
- Défossez, A., Copet, J., Synnaeve, G., and Adi, Y. (2022). High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*.
- Du, Z., Wang, Y., Chen, Q., Shi, X., Lv, X., Zhao, T., Gao, Z., Yang, Y., Gao, C., Wang, H., Yu, F., Liu, H., Sheng, Z., Gu, Y., Deng, C., Wang, W., Zhang, S., Yan, Z., and Zhou, J. (2024). Cosyvoice 2: Scalable streaming speech synthesis with large language models. *arXiv preprint arXiv:2412.10117*.

- Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, pages 6840–6851.
- Kim, J., Kong, J., and Son, J. (2021). Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *International Conference on Machine Learning*, pages 5530–5540.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626.
- Le, M., Vyas, A., Shi, B., Karrer, B., Sari, L., Moritz, R., Williamson, M., Manohar, V., Adi, Y., Mahadeokar, J., and Hsu, W.-N. (2024). Voicebox: Text-guided multilingual universal speech generation at scale. *Advances in Neural Information Processing Systems*.
- Li, N., Liu, S., Liu, Y., Zhao, S., and Liu, M. (2019). Neural speech synthesis with transformer network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 6706–6713.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. (2023). Flow matching for generative modeling. In *International Conference on Learning Representations*.
- Mehta, S., Tu, R., Beskow, J., Székely, É., and Henter, G. E. (2024). Matcha-tts: A fast tts architecture with conditional flow matching. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 11341–11345.
- NVIDIA (2023). Fastertransformer. <https://github.com/NVIDIA/FasterTransformer>.
- Ren, Y., Hu, C., Tan, X., Qin, T., Zhao, S., Zhao, Z., and Liu, T.-Y. (2021). Fastspeech 2: Fast and high-quality end-to-end text to speech. *International Conference on Learning Representations*.
- Ren, Y., Ruan, Y., Tan, X., Qin, T., Zhao, S., Zhao, Z., and Liu, T.-Y. (2019). Fastspeech: Fast, robust and controllable text to speech. In *Advances in Neural Information Processing Systems*.
- Shen, J., Pang, R., Weiss, R. J., Schuster, M., Jaitly, N., Yang, Z., Chen, Z., Zhang, Y., Wang, Y., Skerrv-Ryan, R. J., Saurous, R. A., Agiomyrgiannakis, Y., and Wu, Y. (2018). Natural tts synthesis by conditioning wavenet on mel spectrogram predictions. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 4779–4783.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.
- Wang, C., Chen, S., Wu, Y., Zhang, Z., Zhou, L., Liu, S., Chen, Z., Liu, Y., Wang, H., Li, J., He, L., Zhao, S., and Wei, F. (2023). Neural codec language models are zero-shot text to speech synthesizers. In *arXiv preprint arXiv:2301.02111*.
- Wang, Y., Skerry-Ryan, R. J., Stanton, D., Wu, Y., Weiss, R. J., Jaitly, N., Yang, Z., Xiao, Y., Chen, Z., Bengio, S., Le, Q., Agiomyrgiannakis, Y., Clark, R., and Saurous, R. A. (2017). Tacotron: Towards end-to-end speech synthesis. In *Interspeech*, pages 4006–4010.
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. (2022). Orca: A distributed serving system for transformer-based generative models. In *USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538.
- Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., and Tagliasacchi, M. (2021). Soundstream: An end-to-end neural audio codec. In *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, pages 495–507.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. (2022). Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.